

МОЛДАВСКИЙ ГОСУДАРСТВЕННЫЙ УНИВЕРСИТЕТ

Факультет Математики и Информатики

Департамент Информатики

По предмету Дискретная математика: теория графов.

Лабораторная работа № 3

Тема: Поиск минимального остовного дерева

Лектор: N. Cuciuc

Выполнил: D. Belov

Кишинев, 2026

1. Теория и определения

- **Остовное дерево графа** $G = (V, E)$ - это подграф, который содержит все вершины исходного графа и является деревом (связным графом без циклов).
- **Минимальное остовное дерево** - это остовное дерево взвешенного графа, имеющее минимальный возможный суммарный вес рёбер среди всех возможных остовных деревьев этого графа.
- **Алгоритм Прима** - это жадный алгоритм построения минимального остовного дерева связного взвешенного неориентированного графа. Алгоритм начинает работу со стартовой вершины и постепенно создаёт минимальное остовное дерево, добавляя на каждом шаге по одному ребру минимального веса, связывающему уже построенное дерево с новой вершиной.
- В данной работе используется термин **«обработанные» вершины** (processed). Под этим термином понимается динамическое множество вершин, которые уже включены в состав строящегося минимального остовного дерева.
- Термин **«допустимое ребро»** означает ребро, один конец которого находится в множестве обработанных вершин, а второй - в множестве ещё не посещённых уникальных вершин графа.

2. Описание алгоритма и принцип работы

1. **Инициализация.** На основе ввода пользователя формируется матрица смежности `graph_matrix` фиксированного размера (100 на 100). Отсутствие ребра по умолчанию обозначается значением -1. Все уникальные вершины сохраняются в множестве `unic_points`.
2. **Ввод начальной вершины.** Пользователь явно вводит стартовую вершину. Она становится первым элементом множества обработанных вершин (processed). Результаты (список ребер дерева `ost` и общий вес `total_weight`) изначально пусты.
3. **Основной цикл.** Программа работает до тех пор, пока количество обработанных вершин не сравняется с общим числом уникальных вершин в графе (`len(processed) < len(unic_points)`).
4. **Поиск минимум.** На каждой итерации запускается двойной цикл: внешний перебирает уже обработанные вершины `u`, а внутренний — все доступные вершины графа `v`.
5. **Условие отбора.** Программа ищет ребро (u, v) с минимальным весом, удовлетворяющее трем критериям: Вершина `v` еще не обработана (`v not in processed`). Ребро существует (`weight != -1`). Его вес меньше текущего найденного минимума (`weight < min_weight`).
6. **Проверка на связность.** Если на каком-то шаге допустимое ребро не найдено, но обойдены еще не все вершины, программа сигнализирует о том, что граф является несвязным, и прерывает работу.
7. **Фиксация шага.** Найденное оптимальное ребро добавляется в `ost`, его вес суммируется в `total_weight`, а вершина `v` переходит в разряд обработанных. По завершении цикла выводится итоговая структура дерева.

3. Примеры применения алгоритма и работы программы

Проектирование кабельных и транспортных сетей

В инженерных задачах алгоритм Прима используется для оптимизации прокладки коммуникаций (электрические сети, водопровод, оптоволоконные линии связи, дороги). Решение задачи гарантирует, что все объекты (города, дома, процессоры на плате) будут соединены в единую сеть с минимальными затратами на материалы и бурение/укладку.

Пример работы программы

Входные данные (ввод рёбер):

```
1 2 4
1 3 2
2 3 1
2 4 5
3 4 8
стоп
```

Программа строит матрицу смежности (для уникальных вершин [1, 2, 3, 4]):

ИСХОДНЫЙ ГРАФ (Матрица смежности)

```
  1  2  3  4
1: -1  4  2 -1
2:  4 -1  1  5
3:  2  1 -1  8
4: -1  5  8 -1
```

Вводится стартовая вершина: 1

Пошаговое выполнение алгоритма:

Итерация 1

- processed = {1} (так как является стартовой)
- Ищем ребра с {1} среди необработанных {2, 3, 4}:
 - 1 - 2, вес: 4
 - 1 - 3, вес: 2 - минимальное значение
- Ребро (1, 3, 2) (первая вершина, вторая вершина, вес) добавляется в ost. total_weight = 2, processed становится равным {1, 3}.

Итерация 2

- processed = {1, 3}
- Ищем ребра вершин {1, 3} в необработанных {2, 4}:
 - Из вершины 1: 1 - 2 (вес 4).
 - Из вершины 3: 3 - 2 (вес 1) - минимальное.

- Ребро (3, 2, 1) добавляется в `ost`. `total_weight = 2 + 1 = 3.`, `processed` становится равным {1, 2, 3}.

Итерация 3

- `processed = {1, 2, 3}`
- Ищем ребра из {1, 2, 3} в оставшуюся необработанную вершину {4}:
 - Из вершины 2: 2 - 4 (вес 5) - минимальное
 - Из вершины 3: 3 - 4 (вес 8).
- Ребро (2, 4, 5) добавляется в `ost`. `total_weight = 3 + 5 = 8`
- `processed` становится равным {1, 2, 3, 4}.

Условие выполнения прервано `len(processed) < len(unic_points)`. Цикл завершается.

4. Выводы и оценка сложности

Временная сложность:

В текущей реализации алгоритм делает $O(V \cdot (V \cdot V)) = O(V^3)$ операций в худшем случае, где V - количество уникальных вершин графа.

Пространственная сложность:

Фиксированный размер матрицы смежности $O(N^2)$, где $N = 100$ (выделяется статически под максимальный размер матрицы).

Преимущества текущей реализации:

1. Строгая валидация (Программа перехватывает отрицательные веса рёбер на этапе ввода и проверяет корректность типов данных (try-except), защищая логику от падения).
2. Контроль связности (Наличие проверки `if min_edge is None` предотвращает бесконечное заикливание, если пользователем был введен несвязный граф).
3. Изолированность ввода (Выбор стартовой вершины вынесен в отдельный защищенный цикл, гарантируя контролируемое начало генерации дерева).

5. Листинги программ

```
import sys
```

```
matrix_size = 100
```

```
graph_matrix = [[-1] * matrix_size for _ in range(matrix_size)]
```

```
unic_points = set()
```

```
print("Вводите рёбра в формате: 'вершина1 вершина2 вес' (например, 1 2 23).")
```

```
print("Для окончания ввода напишите 'стоп'.")
```

```

while True:
    user_input = input("Ребро: ").strip()
    if user_input == "стоп":
        break

    try:
        u, v, w = map(int, user_input.split())

        if w < 0:
            print("Ошибка: вес ребра не может быть отрицательным! Попробуйте снова.")
            continue

        graph_matrix[u][v] = w
        graph_matrix[v][u] = w
        unic_points.add(u)
        unic_points.add(v)
    except ValueError:
        print("Неверный формат! Введите три числа через пробел или 'стоп'.")

if not unic_points:
    print("Граф пуст. Выход.")
    sys.exit()

print("\nИСХОДНЫЙ ГРАФ (Матрица смежности)")
sorted_v = sorted(list(unic_points))
print("    " + " ".join(f"{v:3}" for v in sorted_v))
for u in sorted_v:
    row_str = " ".join(f"{graph_matrix[u][v]:3}" for v in sorted_v)
    print(f"{u:3}: {row_str}")
print("-----\n")

while True:
    try:
        start_point = int(input(f"Введите стартовую вершину из доступных {sorted_v}: "))
        if start_point in unic_points:
            break
        print("Такой вершины нет в графе! Попробуйте еще раз.")
    except ValueError:
        print("Введите корректное число.")

processed = {start_point}
ost = []
total_weight = 0

```

```

while len(processed) < len(unic_points):
    min_edge = None
    min_weight = float('inf')

    for u in processed:
        for v in unic_points:
            weight = graph_matrix[u][v]
            if v not in processed and weight != -1 and weight < min_weight:
                min_weight = weight
                min_edge = (u, v, weight)

    if min_edge is None:
        print("\nОшибка: Граф несвязный! Не удалось обойти все вершины.")
        break

    u, v, weight = min_edge
    processed.add(v)
    ost.append((u, v, weight))
    total_weight += weight

print("\n РЕЗУЛЬТАТ (Минимальное остовное дерево)")
for u, v, w in ost:
    print(f"Ребро: {u} - {v}, Вес: {w}")
print(f"Общий вес дерева: {total_weight}")
print("-----")

```